



Disciplina: TEC0790 – Estruturas de Dados
Curso: Técnico em Desenvolvimento de Sistemas

Semana 9 | Listas Sequenciais

Inserções, Remoções Internas e Boas Práticas (TAD)

Objetivos da Semana



Manipulação Interna

Implementar inserção e remoção em posições arbitrárias da lista sequencial através do gerenciamento físico dos dados.



Análise de Desempenho

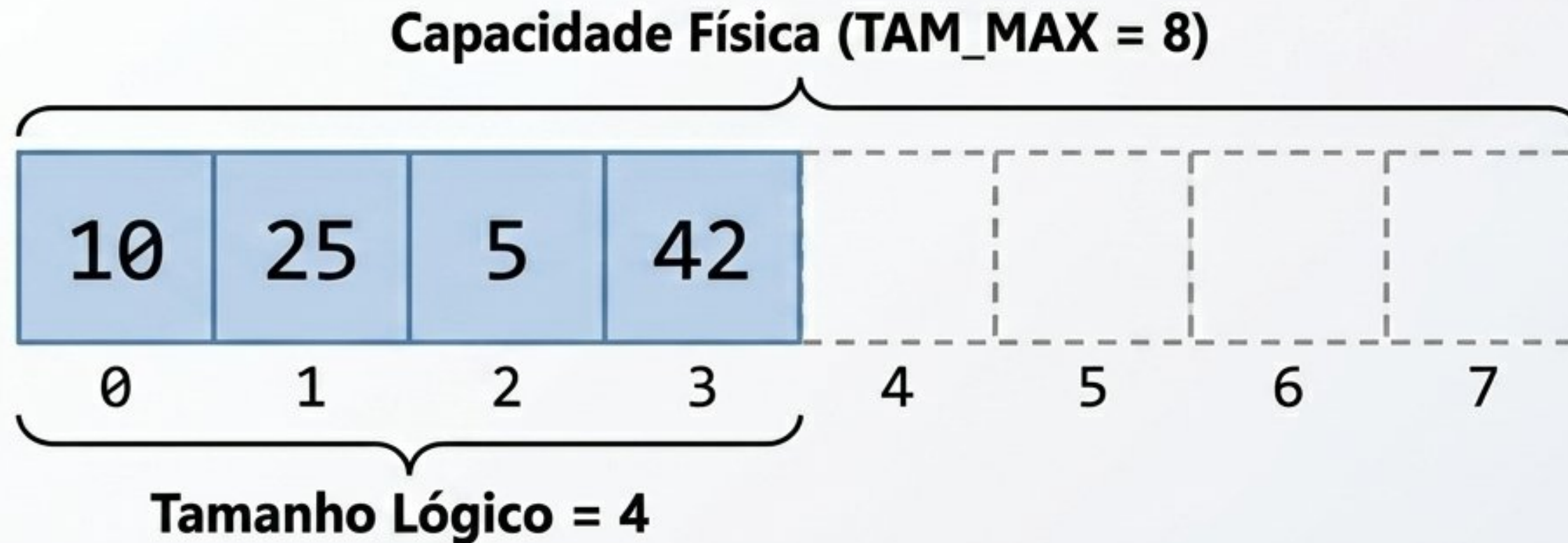
Compreender qualitativamente o impacto computacional do deslocamento de elementos na memória (melhor vs. pior caso).



Encapsulamento (TAD)

Abstrair a lista através de uma struct em C, reunindo dados e variáveis de controle em um Tipo Abstrato de Dados coeso.

Relembrando: Vetor Físico vs. Lista Lógica



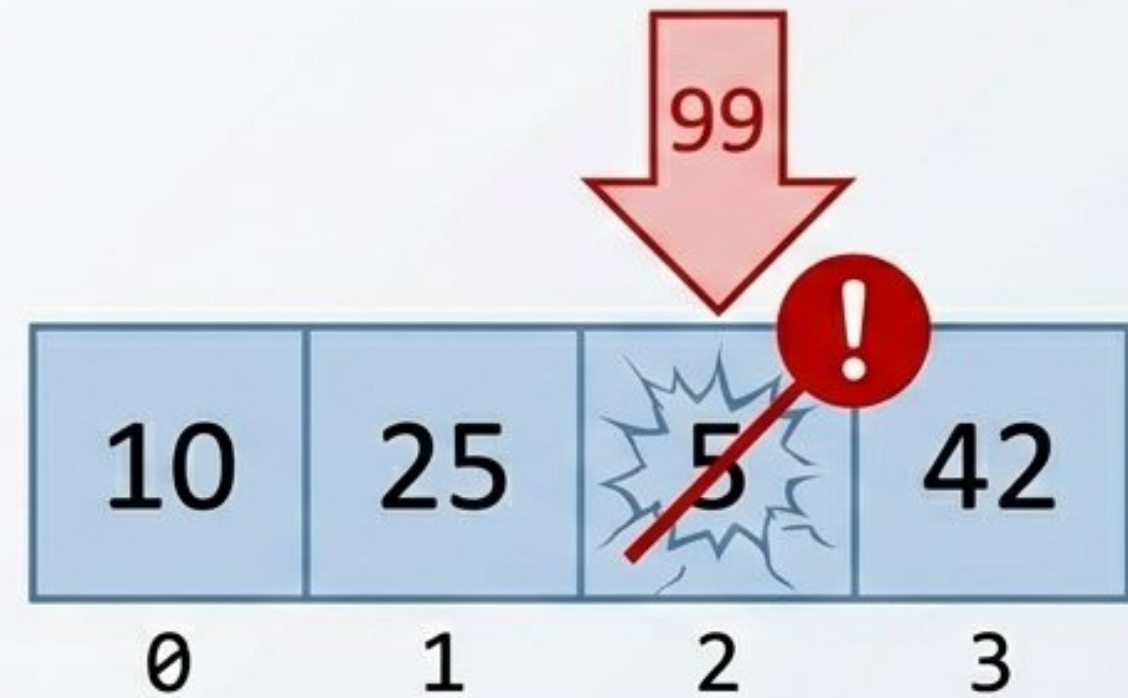
Uma Lista Sequencial não é apenas o vetor físico, mas a união inseparável do Vetor de Dados com a Variável de Controle (tamanho).

- Semana 8: As inserções e remoções ocorreram apenas no final da lista (índice == tamanho).
- Vantagem: A operação no fim é trivial, imediata e não afeta os elementos que já estão na memória.

O Problema das Posições Internas



O que acontece ao inserir ou remover dados no início ou no meio da lista?

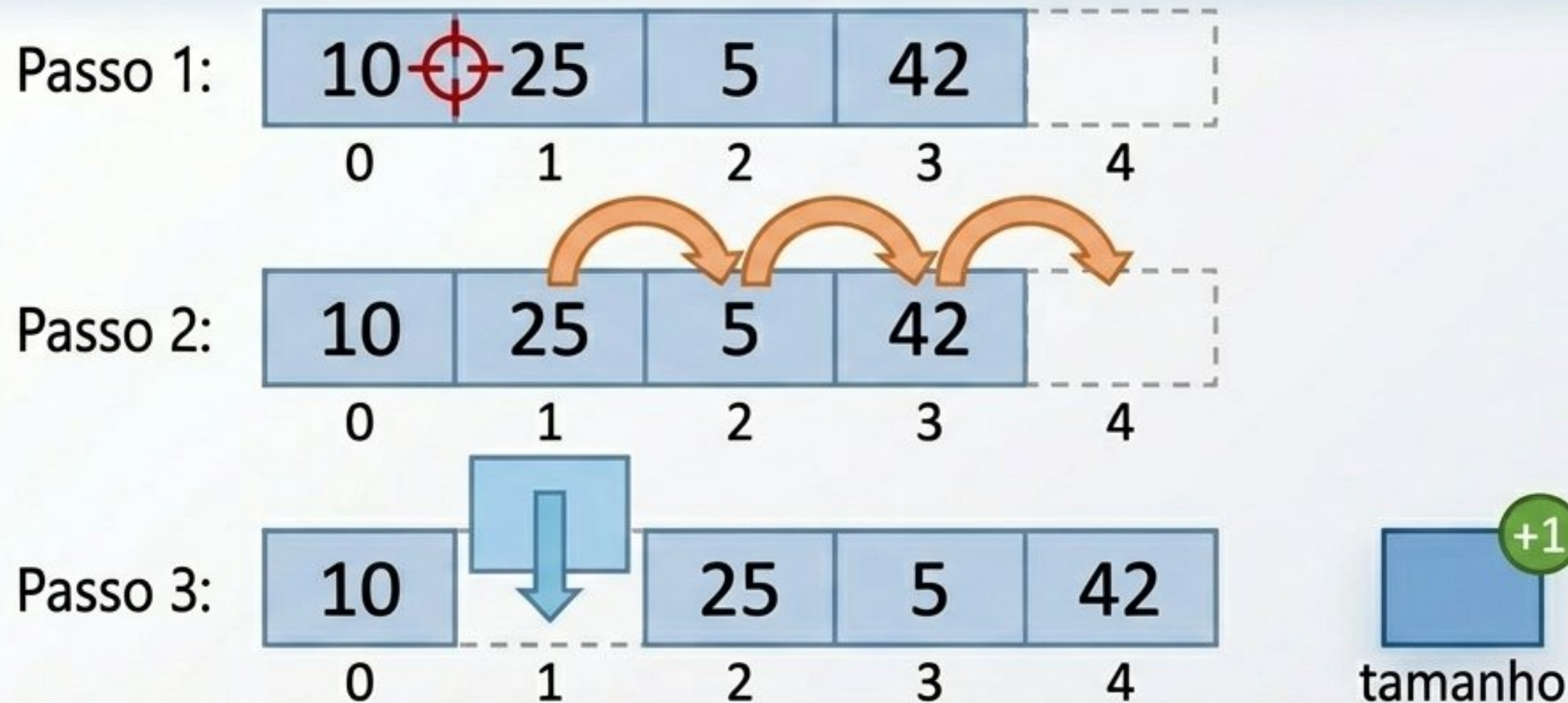


Como vetores ocupam espaços contíguos na memória RAM, não podemos abrir espaço magicamente. Inserir diretamente sobrescreve o dado existente.

Conclusão: A única solução mecânica é realizar o Deslocamento de Elementos físico antes de qualquer alteração lógica.

Lógica de Inserção: Abrindo Espaço Seguro

Regra de Ouro: O deslocamento deve ocorrer estritamente da Direita para a Esquerda.



Aviso Crítico: Se movermos da esquerda para a direita (do alvo para o fim), sobrescreveremos os elementos vizinhos como um efeito dominó accidental!

Pré-requisito: Validar se a lista não está cheia antes de iniciar o processo.

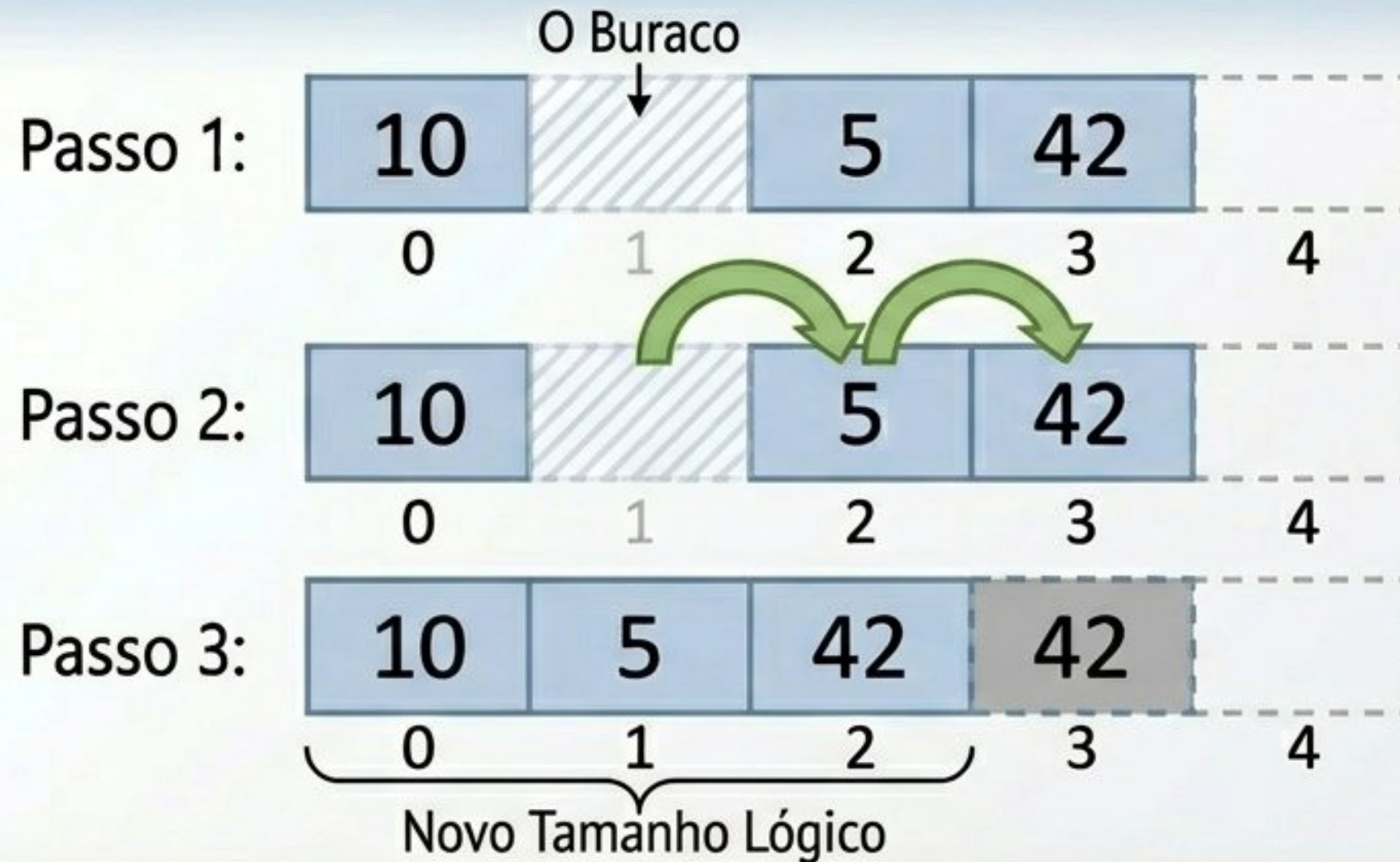
Algoritmo de Inserção: Passo a Passo

- 1 Validar Capacidade:** Verificar se $\text{tamanho} < \text{TAM_MAX}$. Se cheio, abortar.
- 2 Validar Índice:** Verificar se posição é válida (entre 0 e tamanho).
- 3 Deslocamento Reverso:** Laço for. Iniciar no fim físico da lista e copiar dados de $i-1$ para i até o limite da posição.
- 4 Inserir:** Alocar `novo_valor` no índice da posição.
- 5 Atualizar:** Incrementar a variável `tamanho`.

```
// 0 laço crítico de deslocamento
for (int i = tamanho; i > pos; i--) {
    dados[i] = dados[i-1];
}
dados[pos] = novo_valor;
tamanho++;
```


Lógica de Remoção: Fechando o Buraco

Regra de Ouro: O deslocamento ocorre da Esquerda para a Direita.



Insight de Gerenciamento de Memória: Não há comando para apagar o último dado residual da memória. Ao decrementar a variável tamanho, esse lixo de memória passa a ser totalmente ignorado pelas operações da lista.

Pré-requisito: Validar se a lista não está vazia antes de iniciar.

Algoritmo de Remoção: Passo a Passo

- 1 Validar Estado:** Verificar se $\text{tamanho} > 0$. Se vazia, abortar operação.
- 2 Validar Índice:** Garantir que posição está entre 0 e $\text{tamanho} - 1$.
- 3 Capturar Dado (Opcional):** Salvar o valor atual de $\text{dados}[\text{pos}]$ para retornar ao sistema.
- 4 Deslocamento Avançado:** Laço for normal. Iniciar em posição e copiar o vizinho da direita ($i+1$) para o espaço atual (i).
- 5 Atualizar:** Decrementar a variável tamanho.

```
// 0 laço de preenchimento do buraco
for (int i = pos; i < tamanho - 1; i++) {
    dados[i] = dados[i+1];
}

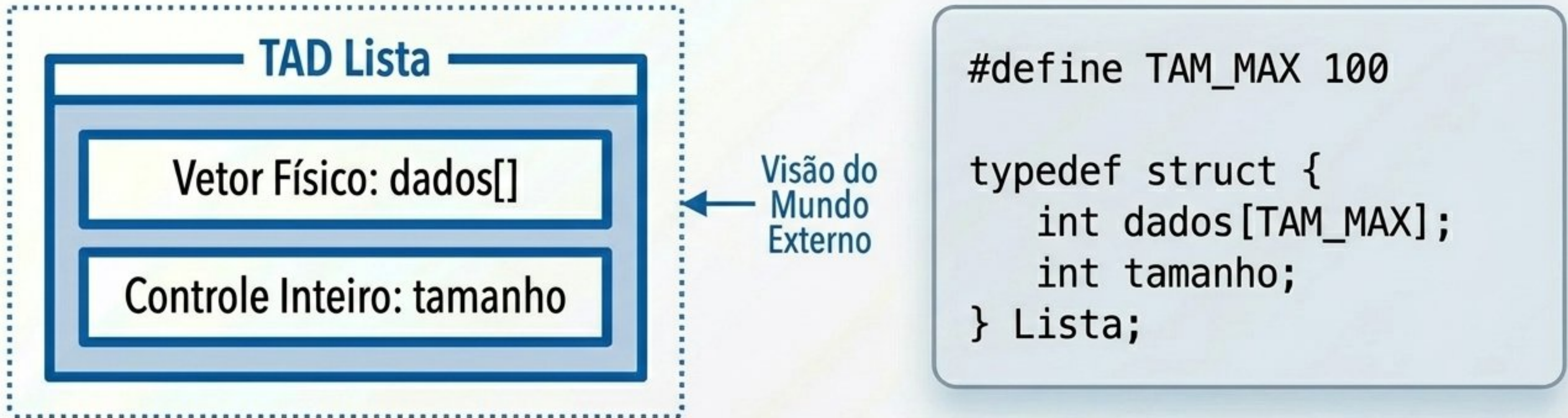
tamanho--;
```


Análise de Desempenho: O Custo do Deslocamento

A eficiência não depende do dado em si, mas de quantos blocos precisam ser fisicamente movidos.



Boas Práticas: Encapsulamento de Dados (TAD)



- **O que é o TAD?:** O Tipo Abstrato de Dados (TAD) agrupa os dados puros e a lógica de controle em uma unidade coesa.
- Em C, unimos o vetor físico e o contador de controle utilizando a struct.
- **Vantagem:** A lista inteira pode trafegar pelas funções utilizando um único parâmetro de ponteiro (Lista *l). A integridade interna é preservada.

A Interface de Uso: Funções do TAD Lista



Os sistemas externos interagem com a lista exclusivamente através destas funções prototipadas. Modificar o campo tamanho manualmente quebra a integridade da lista.

Construção

<code>void inicializar(Lista *l);</code>	(Zera o controle lógico de tamanho)
--	-------------------------------------

Manipulação

<code>int inserir_pos(Lista *l, int pos, int valor);</code>	(Maneja limites físicos e deslocamentos à direita)
---	--

<code>int remover_pos(Lista *l, int pos, int *removido);</code>	(Maneja deslocamentos à esquerda e encolhe a lista)
---	---

Consulta

<code>int buscar(Lista *l, int valor);</code>
<code>void imprimir(Lista *l);</code>

Estudo de Caso: Cadastro Real de Produtos

O Cenário: Sistemas de ERP e gestão raramente listam tipos primitivos puros (int). A evolução exige que o vetor armazene blocos completos de registros estruturados.

```
// A Entidade  
typedef struct {  
    int codigo;  
    char desc[50];  
    float preco;  
} Produto;
```



```
// A Lista Adaptada  
typedef struct {  
    Produto dados[TAM_MAX];  
    int tamanho;  
} Lista;
```



Gerenciador
de Estoque

O Insight Físico: A lógica complexa dos laços for de deslocamento permanece exatamente a mesma. O hardware apenas moverá blocos de bytes maiores na memória em cada iteração.

Atividade Prática: Codificação em Laboratório

Missão: Abandonar o uso de vetores soltos e implementar um Sistema de Gerenciamento de Produtos fundado no TAD Lista construído do zero.

- ☐ **Tarefa 1:** Fundação. Declarar as estruturas (struct Produto e struct Lista).
- ☐ **Tarefa 2:** Estado Zero. Criar a função inicializar que define a lista segura como vazia.
- ☐ **Tarefa 3:** Engenharia Interna. Programar as funções de inserção e remoção em posições arbitrárias. Foco absoluto na direção correta dos laços for de deslocamento.
- ☐ **Tarefa 4:** Interface Cliente. Estruturar a função main() para instanciar a Lista e demonstrar a inserção de três produtos em ordens diferentes.



Desafio Final (Verificação de Aprendizagem)



Challenge Card

Objetivo Prático: Construir robustez defensiva em uma função crítica de manipulação de memória.

A Tarefa: Implementar uma função dedicada à remoção de dados baseada estritamente na posição que o usuário digitar no console.

Requisitos Obrigatórios (Critérios de Avaliação):

- [1] **Captura:** Ler de forma segura a posição solicitada (o índice) pelo usuário final.
- [2] **Integridade:** Garantir o reajuste mecânico imediato (deslocamento) de todos os elementos subsequentes para fechar o buraco.
- [3] **Tratamento Defensivo:** Tratar rigorosamente a inserção de índices inválidos. O sistema deve impedir que o for tente acessar posições menores que 0 ou maiores que o tamanho da lista, exibindo uma mensagem de erro adequada.